

Chat application

Team ID : GEN_001

Team members:

Name:

ID:

نوران اسامه محمد على

2023170680

عثمان ناصر محمد حسن

2023170365

سهيلة خالد عبد الرشيد محمد

2023170280

سما أيمن منير محمد

2023170267

عبدالعزيز عادل محمد حامد

2023170352

جني وليد سعيد فهمي

2023170166

Project Overview

The application enables users to engage in private and group chats, manage contacts, and share stories with text and images. It utilizes JSON files for data persistence, storing user profiles, chat histories, and story details. The project demonstrates the use of object-oriented programming and data structures like maps, sets, vectors, and classes to create a functional and modular chat system tailored for academic evaluation.

Objectives

- Enable users to register, log in, and manage their profiles.
- Support private and group chat rooms with message status tracking (seen/unseen).
- Allow users to post stories with text and images, visible to selected contacts.
- Provide a user-friendly interface using Qt Widgets for seamless interaction.
- Store and retrieve user data, chat history, and stories using JSON files.

Class Descriptions

Story Class

- **File:** Story.h, Story.cpp
- **Purpose:** Manages user stories, including content, images, and viewer tracking.
- **Attributes:**
 - **userID:** Integer identifying the user who posted the story.
 - **content:** String containing the story text.
 - **photoPath:** String path to the story image.
 - **publishedTime:** String representing the time the story was posted.
 - **available:** Boolean indicating if the story is still viewable.
 - **viewers:** Vector of integers listing user IDs who viewed the story.
- **Constructors:**
 - **Default constructor:** Initializes an empty story.
 - **Parameterized constructor:** Initializes a story with userID, publishedTime, content, and photoPath.

- **Usage:** Stories are created when users post content, stored in userData.json, and displayed to contacts based on visibility settings.

User Class

- **File:** User.h, User.cpp
- **Purpose:** Represents a user in the system, managing their profile, contacts, chat rooms, and stories.
- **Attributes:**
 - id: Integer uniquely identifying the user.
 - number: Integer representing the user's phone number.
 - password: String storing the user's password.
 - userContacts: Map of integers to strings, mapping contact numbers to names.
 - Fname, Lname: Strings for the user's first and last names.
 - avatar: String path to the user's profile picture.
 - lastSeen: String indicating the user's last online status.
 - chatRooms: Vector of ChatRoom objects representing the user's chat rooms.

- profile: ProfileDescription object containing the user's bio and visibility settings.
- story: Story object representing the user's current story.

- **Methods:**

- getNameFromContacts(int): Retrieves a contact's name by their number.
- getID(), getNumber(), getPassword(): Getters for user attributes.
- setID(int), setNumber(int), setPassword(string): Setters for user attributes.
- getContacts(): Returns the user's contact list.
- addContacts(int, string): Adds a contact to the user's list.
- setContacts(map<int, string>): Sets the entire contact list.
- addChatRoom(ChatRoom): Adds a chat room to the user's list.

- **Usage:** The User class is central to the application, managing user interactions, authentication, and data persistence.

Chat Room Class

- **File:** ChatRoom.h, ChatRoom.cpp
- **Purpose:** Manages chat rooms, including group or private chats, tracking participants, messages, and view status.
- **Attributes:**
 - chatRoomID: Integer identifying the chat room.
 - userIDs: Vector of integers listing user IDs participating in the chat.
 - messages: Vector of Message objects containing the chat's messages.
 - viewedBy: Unordered set of integers tracking user IDs who have viewed the chat.
 - gcAvatar: String representing the group chat's avatar (if applicable).
 - gcName: String representing the group chat's name (if applicable).
- **Constructors:**
 - **Default constructor:** Initializes an empty chat room with chatRoomID set to 0, empty userIDs, and empty messages.
 - **Parameterized constructor:** Initializes a chat room with a specified chatRoomID and userIDs.

- **Methods:**

- `setChatRoomID(int newid)`: Sets the chat room ID to the specified value.
- `addMesaages(const Message& m)`: Adds a message to the chat room's message vector
- `getChatRoomID() const`: Returns the chat room ID.
- `getUserIDs() const`: Returns the vector of user IDs.
- `getMessages() const`: Returns a const reference to the vector of messages.
- `setMessages(const vector<Message>& msgs)`: Sets the chat room's messages to the provided vector.
- `type() const`: Returns "Group" if more than two users are in the chat, otherwise "Private".
- `maxMessID()`: Returns the highest message ID in the chat room's messages.
- `removeMessage(int messageId)`: Removes the message with the specified message ID from the chat room's messages.

- **Usage:** Chat rooms are created to manage private or group conversations, storing messages and tracking participants. Data is likely stored in a file like `userData.json` (based on Story class usage) and used to display chats to users.

Message Class

- **File:** Message.h, Message.cpp
- **Purpose:** Manages individual messages in a chat, including text, sender, status, and view tracking.
- **Attributes:**
 - **messageID:** Integer identifying the message.
 - **userIDs:** Integer identifying the user who sent the message.
 - **viewedBy:** Vector of integers listing user IDs who have viewed the message.
 - **text:** String containing the message content.
 - **status:** String indicating the message status (e.g., “unseen”).
 - **dateSent:** String representing the date the message was sent (format: yyyy-MM-dd).
 - **timeSent:** String representing the time the message was sent (format: HH:mm).
- **Constructors:**
 - **Default constructor:** Initializes an empty message with messageID and userID set to 0, empty text, status as “unseen”, and empty dateSent and timeSent.
 - **Parameterized constructor:** Initializes a chat room with a specified chatRoomID and userIDs.
- **Methods:**

- `getMessageID()` const: Returns the message ID.
- `getUsedID()` const: Returns the user ID of the sender.
- `setStatus(const string& newStatus)`: Sets the message status to the specified value.
- `getStatus()` const: Returns the current message status.
- `getViewedBy()` const: Returns a const reference to the vector of user IDs who viewed the message.
- `addViewer(int userId)`: Adds a user ID to the viewedBy vector if not already present.
- `setViewedBy(const vector& viewers)`: Sets the viewedBy vector to the provided list of user IDs.
- **Usage** Messages are created when users send text in a chat, stored in `userData.json`, and displayed in chat rooms with visibility and status tracking for viewers.

Profile Description Class

- **File:** `ProfileDescription.h`, `ProfileDescription.cpp`
- **Purpose:** Manages a user's profile description, including the "about" text and visibility settings.
- **Attributes:**
 - `about`: String containing the user's profile description text.

- visibility: Boolean indicating whether the profile description is publicly viewable.
- **Constructors:**
 - **Default constructor:** Initializes an empty profile description with an empty “about” string and visibility set to false.
 - **Parameterized constructor:** Initializes a profile description with a specified userID, “about” text, and optional visibility (defaults to false).
- **Usage:** Profile descriptions are created to store and display a user’s “about” information, likely stored in userData.json, and shown to other users based on visibility settings.

JSON Data Structure

userData.json

- **Purpose:** Primary data storage for user information, chat rooms, and stories.
- **Structure:**
 - Key: User ID (string).
 - Value: Object containing:
 - Fname, Lname: User's first and last names.
 - avatar: Path to the user's profile picture.

- chatRooms: Array of chat room objects, each with:
 - chatRoomID: Unique identifier.
 - messages: Array of message objects (see Message class).
 - type: "Private" or "Group".
 - userIDs: Array of user IDs in the chat room.
 - groupName, groupAvatar: Group-specific metadata (for group chats).
- contacts: Object mapping contact numbers to names.
- id: User ID (integer).
- lastSeen: Online status or timestamp.
- number: Phone number.
- password: User password.
- profile: Object with about (bio) and visibility (boolean).
- stories: Object with story details (see Story class).

• Example:

```
{
  "1": {
    "Fname": "abdelaziz",
    "Lname": "adel",
    "avatar": "icons/abdelaziz.jpg",
    "chatRooms": [...],
    "contacts": {
      "1099186239": "Sama Ayman",
      "1234567890": "Jana waleed"
    },
    "id": 1,
    "lastSeen": "online",
    "number": 1234578792,
    "password": "abc123",
    "profile": {
      "about": "one of the stupidest people!!!",
      "visibility": true
    },
    "stories": {
      "photoPath": "icons/story1.jpeg",
      "publishedTime": "",
      "storyText": "عظيم الشروق",
      "userID": 1,
      "viewers_id": [3]
    }
  }
}
```

Input and Output Scenarios

Below are detailed scenarios for key functions in the DS Project, describing the inputs, processing steps, and outputs. These scenarios are derived from the User and Story classes and the JSON data structures.

1. Adding a Contact (User::addContacts)

- **Function:** void User::addContacts(int n, string s)
- **Input:**
 - n: Integer representing the contact's phone number (e.g., 1099186239).
 - s: String representing the contact's name (e.g., "Sama Ayman").
- **Process:**
 1. The function receives the phone number and name as parameters.
 2. Inserts the pair into the userContacts map, associating the number with the name.
 3. Updates the user's data in userData.json to persist the new contact.

- **Output:**

- The userContacts map is updated with the new entry.
- Example: For user ID 1, userContacts now includes {1099186239: "Sama Ayman"}.
- JSON update in userData.json:
- "contacts": {
 "1099186239": "Sama Ayman"
}

- **Scenario Example:**

- **Input:** n = 1234567890, s = "Jana waleed"
- **Process:** Add {1234567890: "Jana waleed"} to userContacts and save to userData.json.
- **Output:** User ID 1's contact list now includes "Jana waleed" for number 1234567890.

2. Retrieving Contact Name

(User::getNameFromContacts)

- **Function:** string User::getNameFromContacts(int number)
- **Input:**

- number: Integer representing the contact's phone number (e.g., 1099186239).

- **Process:**

1. Iterates through the userContacts map to find the entry with the matching number.
2. If found, returns the associated name; otherwise, returns an empty string.

- **Output:**

- String containing the contact's name or an empty string if not found.
- Example: For number = 1099186239, returns "Sama Ayman".

- **Scenario Example:**

- **Input:** number = 1234578792
- **Process:** Search userContacts of user ID 3, find {1234578792: "Abdelaziz Adel"}.
- **Output:** Returns "Abdelaziz Adel".

3. Creating a Story (Story::Story Constructor)

- **Function:** Story::Story(int userID, string publishedTime, string content, string photopath)
- **Input:**

- userID: Integer identifying the user (e.g., 1).
- publishedTime: String representing the posting time (e.g., "2025-05-16 17:00:00").
- content: String containing the story text (e.g., "عظيم الشروق").
- photopath: String path to the story image (e.g., "icons/story1.jpeg").

- **Process:**

1. Initializes a Story object with the provided userID, publishedTime, content, and photopath.
2. Sets available to true and initializes an empty viewers vector.
3. Updates the user's stories field in userData.json with the new story data.

- **Output:**

- A Story object is created and stored in the user's data.
- JSON update in userData.json:
 - "stories": {
 - "photoPath": "icons/story1.jpeg",
 - "publishedTime": "2025-05-16 17:00:00",


```
"storyText": "عظيم الشروق",  
"userID": 1,  
"viewers_id": []  
}
```

- **Scenario Example:**

- **Input:** userID = 3, publishedTime = "2025-05-16 17:00:00", content = "hellooo!", photopath = "icons/story3.jpeg"
- **Process:** Create a Story object and update userData.json for user ID 3.
- **Output:** User ID 3's story is updated with the new content and image.

4. Adding a Chat Room (User::addChatRoom)

- **Function:** void User::addChatRoom(ChatRoom c)

- **Input:**

- c: ChatRoom object containing:
 - chatRoomID: Unique identifier (e.g., 100).
 - type: String ("Private" or "Group").
 - userIDs: Vector of user IDs (e.g., [1, 3]).

- messages: Vector of Message objects (initially empty).
- groupName, groupAvatar: Strings for group chats (e.g., "DS", "icons/gc2.jpg").

- **Process:**

1. Appends the ChatRoom object to the user's chatRooms vector.
2. Updates userData.json to include the new chat room in the user's data.

- **Output:**

- The user's chatRooms vector includes the new chat room.
- JSON update in userData.json:
- "chatRooms": [{
 "chatRoomID": 100,
 "messages": [],
 "type": "Private",
 "userIDs": [1, 3]
 }
]

- **Scenario Example:**

- **Input:** ChatRoom with chatRoomID = 109, type = "Group", userIDs = [1, 2, 4, 6, 7, 3], groupName = "DS", groupAvatar = "icons/gc2.jpg"
- **Process:** Add the chat room to user ID 1's chatRooms and update userData.json.
- **Output:** User ID 1's chat rooms now include the "DS" group chat.

5. Sending a Message (Assumed ChatRoom Function)

- **Function:** Assumed ChatRoom::sendMessage (not provided, inferred from JSON).
- **Input:**
 - userID: Integer of the sender (e.g., 3).
 - text: String message content (e.g., "um... hi").